

vit^{API}

vit: lay out whole pages in Typst, mark what should morph, and let the browser do the rest.

The identity channel is a **name**: the page records what it marked and where the layout put it, and the deck draws each region again in a frame of its own, on an element carrying data-vit-key. That element is the boundary, and it is a CSS box, which is what the browser needs to morph it.

There are three kinds of motion. A **transition** is a change of layout state (page to page, frame to frame) and runs on the View Transitions API. An **element animation** is one object varying with a parameter (mark with several states, stepped with -) and runs on Web Animations. A **continuous animation** is an object moving on its own, without keys. Both of those are tween's and are imported from it; vit itself handles the first. To tell them apart, ask whether the layout changed or one object is moving. Design trade-offs and browser pitfalls are in the README.

vit

- turning()
- mark()
- veil()
- anchor()
- player()
- deck()
- reveal()
- layers()
- build()
- slide()

Variables

- version

turning

How the pages under this turn, for a run of them rather than the whole deck: #show: turning("push"), and every page after it in that scope turns that way unless it says otherwise itself. Takes what deck(transition:) takes.

```
#show: turning(none) // a section that does not move
```

Parameters

```
turning(transition: none | str | dictionary) -> function
```

transition `none` or `str` or dictionary

A name from transitions, a pair (enter:, leave:), a dictionary with settings, or none.

mark

Give a piece of content a name: this is one object, and the same key on two adjacent pages is the same object, which the browser pairs and interpolates: position, size, colour, rotation. This is a **transition**: the layout changed. A mark only gives identity between pages. Motion **inside** it (N states of one drawing, or the drawing playing them over time) belongs to tween, needs no mark, and is written where the drawing is.

In a formula, pass the term as an equation, \$ #mark("sq")(\$x^2\$) + #mark("lin")(\$b x\$) = c \$, not as bare math: a mark boxes what it is given, and a box lays its content out as markup, which would set b x upright in the body font. An equation inside an equation is transparent, so the term is typeset exactly as it would be unmarked, and the operators around it keep their spacing. What is left unmarked is the page's: a and = cross-fade with it, which is invisible where they stay put and a double image where they move.

What has no in-between (text that changes, a path against an arc, states whose element count differs) cross-fades. The PDF has no player and shows the last state (the finished figure, like a handout). In the HTML the other states are stacked on the first state's box (place, taking no layout space) and hidden by the runtime as data.

With tween(play:) the same states are played over time instead of stepped; the drawing says so itself and the deck only carries the word.

The inner box is required: what says where the region is and how big it is is a pair of points placed in its two corners, and only something with corners can hold them. Of everything that can stand in a paragraph that is box and block; which of the two is block: below. Between them, block is out as the default: it is block-level, and a mark has to work on a word inside a sentence and on a term inside a formula; it is also breakable, and a mark split over two lines has no single box to interpolate.

A box does wrap its content, like ordinary text. But it is atomic, so one that does not fit the rest of the line moves to a line of its own: marking a word, a title, a term or a figure costs nothing, while marking a whole sentence in the middle of running prose reflows the paragraph around it.

Parameters

```
mark({
  key: str,
  transition: none | str | dictionary,
  stack: none | array,
  block: bool,
  body: content
}) -> content
```

key `str`

The name. Same name on two adjacent pages = one pair. Letters, digits, _ and - (it becomes a CSS view-transition-name).

transition `none` or `str` or dictionary

This object's own enter/leave effect: a name from transitions, the same both ways, or (enter: name, leave: name), where enter is how it appears and leave how it disappears (going back replays either in reverse). It applies only when the mark is one-sided in a transition; a paired mark morphs regardless. "wipe-up" reveals from the bottom edge up, "slide" pushes in from the right (by its own width, fading as it goes), "zoom" shrinks into place, "none" appears at once. Unset (none) folds the mark into the page: within a page it cross-fades with the layout, between pages it pushes, wipes or fades with the whole page. Definition, theorem and proof each revealed from a different edge is three marks with one value each. One object has one effect: given on any occurrence of the key, it holds for all of them, and two occurrences may not disagree. Marks nest: one around an assembly is a group of its own, and the marks inside it keep their own identity. The group carries what is left once they are lifted out, which is how the first change's result takes part in the next one as a whole (see fit and anchor in transitions). Settings ride along in the same dictionary (duration, easing, zoom, push; see transitions) and apply to this mark alone, whatever pace the page keeps.

Default: `none`

stack `none` or array

Which stack of layers this is one of, as layers says: (key, index). The deck aligns a stack by the point its layers share, and everything drawn inside a layer moves with it.

Default: `none`

block `bool`

How the identity is held: in a box, or in a block.

A box by default, and that is what lets a mark sit inside a sentence or a formula. But a box is only as tall as the glyphs in it, where a line is as tall as the line. So content that is a block in its own right (a title in a header band, a figure, a column) measures differently boxed than it did unmarked, and anything that lays it out by measuring it (fitting it to a width, centring it on the horizon) moves. Marking such content with block: true holds the identity in an unbreakable block instead, which measures exactly as the content did, and the layout does not notice the mark at all. It is block-level, so it is wrong inside a sentence, which is why it is not the default.

Default: `false`

body `content`

What the object is. Several states of one drawing are tween's, not a mark's: mark(key, tween(s0, s1, ...)). To hide a mark, veil rather than hide: the deck gives every region an element of its own, and a bare hide leaves it giving one to a region the page meant to drop.

veil

hide, for content with marks in it. Hidden content keeps its place in the layout and leaves no ink, so a mark inside it has a position and nothing to show, and the deck must not give it an element of its own: an empty one would still be named, and still morph. Typst should not be asked whether something is hidden, so this says so as it hides, as a style the marks inside read (tween's hidden, since a drawing may want to know too).

Parameters

```
veil(body: content) -> content
```

anchor

A named point, for layers to align a stack by. The body keeps its place in the layout and leaves no ink, and the corner it starts at is what the name stands for, so the point may be anywhere in the picture rather than at a corner of it or in the middle.

A stack of layers is one drawing at several stages, and the later stage is usually drawn with the earlier one's parts still in it, hidden, so that both are aligned alike. Naming one of those parts in both layers is what lets the deck put them together: the layers meet at that point, and everything else follows from the layout.

layers("pb", diagram-a, anchor-in-b-drawn-hidden)

A mark is a named point too, at the corner its box starts at, so two layers that already share a marked object need nothing else.

Parameters

```
anchor({
  key: str,
  body: content
}) -> content
```

key `str`

The name. The same name in two layers is the point they share.

body `content`

What holds the place: usually the part the later layer redraws.

player

The player: the stylesheet, the runtime and the chrome (the toolbar, the overview, the speaker view, the laser) around pages that are already laid out. This is all of vit's side of a deck. What a page is made of, how big it is and what colour it sits on are the document's, and the player reads them off the page it is on rather than being told twice.

deck is a document that is nothing but a vit deck: it sets the page, the text and the title, and then calls this. A package that already is the document (one that lays out its own pages, with its own margins, header and footer) sets its page as it always did and calls this directly. Either way there is one page, set once, and slide cuts its frames to it.

Parameters

```
player({
  width: length,
  height: length,
  margin: length | dictionary,
  fill: color,
  pdf: auto | none | str,
  duration: int,
  easing: array,
  transition: none | str | dictionary,
  theme: auto | str,
  laser: dictionary,
  body: content
}) -> content
```

width `length`

Layout width, the size of a page. The PDF's page is this wide and so is every frame's html.frame, and the player's aspect ratio follows.

Default: `1280pt`

height `length`

Layout height.

Default: `720pt`

margin `length` or dictionary

How far the layout is held off the edge of a page. slide reads it back as the inset of every frame, so a document that composes its own margins inside the pages it hands over sets 0pt here and is framed exactly.

Default: `60pt`

fill `color`

Layout background. html.frame carries no page background, so this opens the stylesheet as --vit-page and is painted under the slides and the thumbnails; on the PDF side deck gives the same value to page(fill:), so the two sides match.

Default: `rgb("#111318")`

pdf `auto` or `none` or `str`

Target of the toolbar's PDF download link. auto = the .pdf with the HTML's name, none = no button, a string is used as is.

Default: `auto`

duration `int`

Transition duration in milliseconds. Adjustable live with - / = / 0.

Default: `700`

easing `array`

Transition easing: the two control points of a cubic Bézier, as in CSS cubic-bezier(x1, y1, x2, y2). (0, 0, 1, 1) is linear. Used for page transitions and element-animation steps alike.

Default: `(0.32, 0.72, 0, 1)`

transition `none` or `str` or dictionary

The default **page-to-page** transition: how unmarked content enters and leaves when turning to another page: a name from transitions, the same both ways, Or (enter: name, leave: name), with enter for the page being entered leave for the page being left. Paired marks morph regardless; one-sided marks without an effect of their own fold into the page. The effect "none" switches that side at once while the transition runs (so marks still morph); none runs no transition between pages at all. Not used between frames of one page: there the layout stays put and only changes incrementally, so root cross-fades and elements enter and leave by mark(transition:). A single page can override it with slide(transition:). The dictionary may carry settings for this transition as well (duration, easing, zoom, push); see transitions.

Default: `"fade"`

theme `auto` or `str`

Light/dark theme of the player chrome (toolbar, overview, speaker view). auto follows the system, "dark" / "Light" fix it. Chrome only; the layout's colours are Typst's.

Default: `auto`

laser `dictionary`

What the laser pointer is when a deck is opened: ink, the colour of the dot and of the tracer behind it, size, how wide the dot is, and trail, how long the tracer lasts, in milliseconds (0 for none). The presenter can change all three from the settings panel (,) and their millisecond remembers what they chose. This only sets where a deck starts.

Default: (ink: `rgb("#ff3c3c")`, size: `32pt`, trail: `400`)

body `content`

The pages.

deck

A document that is a deck: the page it is laid out on, the text it is set in, the title it goes by, and then the player around it. One compile gives the PDF, one with --features html the HTML; side by side under one name, the toolbar's download link finds the PDF.

#show: deck.with(title: "-"). Everything the player takes may be named here too and is passed straight on. The arguments only deck takes are the ones that make it a document rather than a player.

Parameters

```
deck({
  title: str,
  width: length,
  height: length,
  margin: length | dictionary,
  fill: color,
  font: array,
  ..rest: arguments,
  body: content
}) -> content
```

title `str`

Document title.

Default: `"vit"`

width `length`

Layout width. The page's size on the PDF side, and the player's layout on the HTML side: one figure, given once, and both sides take it from here.

Default: `1280pt`

height `length`

Layout height.

Default: `720pt`

margin `length` or dictionary

How far the layout is held off the edge of the page: the PDF's page margin and every frame's inset alike, from this one figure.

Default: `60pt`

fill `color`

Color background: page(fill:) on the PDF side, and the ground the stylesheet paints under the slides and thumbnails on the HTML side: identical on both, and independent of the chrome theme.

Default: `rgb("#111318")`

font `array`

Font stack, glyph-by-glyph fallback in order. The default carries a CJK family for a reason: with a Latin family alone, CJK text falls back glyph by glyph to whatever system family has the glyph, and weights differ within one line.

Default: `("DejaVu Sans", "Noto Sans CJK SC")`

..rest `arguments`

The player's settings, handed on untouched.

body `content`

The pages.

reveal

Frames from one description, written once. reveal(3, (step, at) => ...) renders the body once per frame (step is which frame it is, 1 to n), and at(k, x) is x from frame k on. Before then it holds x's space if x is content, so the layout never moves and nothing jumps, and is none otherwise, which is how a stroke or a fill is centered off:

```
slide(..reveal(2, (step, at) => diagram({
  node((0, 0), $A$)
  node((1, 0), at(2, mark("b"), transition: "zoom")[$B$]))
  edge((0, 0), (1, 0), at(2, $f$), "->", stroke: at(2, 1pt))
}))
```

Everything is written where it belongs, each part says when it arrives, and because what has not arrived still takes its space, every frame is laid out identically. That is what lets the marks morph instead of the page re-flowing under them.

Parameters

```
reveal({
  n: int,
  body: function
}) -> array
```

n `int`

How many frames.

body `function`

Called once per frame with the frame's number and at.

layers

Layers of one picture, hung from a point they share. Each layer is a mark of its own (key-1, key-2, ...), so a layer that is on two frames keeps its identity: if a later layer makes the picture bigger, the earlier one glides to where it now sits instead of being redrawn. The last layer sizes the stack and the ones before it move onto the shared point, a none layer is left out, which is how one arrives:

layers("pb", the-square, if step == 3 { the-circle })

What holds the layers together is a point named in both: anchor("-") around a part that both draw, or the same mark in both. The point may be anywhere in the picture rather than at a corner of it or in the middle, because what this works from is the point's offset **inside** its own layer, which the layout knows wherever it sits. A layer that shares no point with the last one says so at compile time, and names what to do about it.

A layer that is a drawing keeps its own picture only if the package lays it out the same way each time, so a later layer that must reach into an earlier one draws that one's anchors hidden.

Parameters

```
layers({
  key: str,
  ..parts: content | none
}) -> content
```

key `str`

Names the stack; each layer's mark key is this and its place in it.

..parts `content` or `none`

The layers, back to front.

build

Frames that accumulate. build(a, b, c) is three frames (a, then a and b, then all three), so what stays on the page is written once and the source reads as what each step adds:

```
slide(title: "Cauchy sequences", ..build(
  definition,
  {#(14pt) #theorem},
  {#(14pt) #proof},
))
```

Content in, content out, so it builds the body of a drawing just as well: ..build(cospan, square, cone).map(diagram).

Parameters

```
build(..parts: content) -> array
```

..parts `content`

The parts, in the order they appear.

slide

One page. On the HTML side a whole-page html.frame (hence pixel-identical to the PDF); on the PDF side a page.

Several bodies are **frames of the same page**: navigation walks them one by one, the overview merges them into one thumbnail with dots for the positions. Frames still transition with View Transitions, so "the second frame has one more line" is an element-level interpolation, not a page jump.

```
#slide(title: "Two frames")[first][first + something added]
```

Parameters

```
slide({
  title: content | str | none,
  note: content | str | none,
  transition: none | str | dictionary,
  ..frames: content
}) -> content
```

title `content` or `str` or `none`

Shown only in the thumbnail caption, never in the layout. May be content.

Default: `none`

note `content` or `str` or `none`

Speaker notes. HTML only, placed in the page's <aside class="vit-note"> (not in the layout, not in the PDF) and read by the desk the deck opens on and by the speaker view (s). May be content: paragraphs, lists, emphasis all render.

Default: `none`

transition `none` or `str` or dictionary

Overrides deck(transition:) for this page: a name from transitions, the same both ways, or (enter: name, leave: name), where enter is how this page comes in when turning to it and leave how the page before it goes out, with settings of its own if it wants them (duration, easing, zoom, push; see transitions). Not used between frames of this page; individual elements enter and leave by mark(transition:). Going back, the page being left decides, so a transition always replays in reverse. None takes the decks.

Default: `none`

..frames `content`

The frames of this page. None at all is one blank page.

version

The package's version, read from the manifest so there is one place to bump it. Not named version: that is a Typst built-in, and import: * would shadow it.